

Name: Praisel Richard Ekpenyong

Email: p.ekpenyong8162@miva.edu.ng

Department: Cybersecurity

Matric Number: 2025/A/CYB/0205

Source Files: https://github.com/praisel-ekpenyong/COS201_Student_Course_Management_System

COS201 - Computer Programming 1 - Lab Assessment

1. What the System Does

The Student Course Management System is a Java application that helps a student record and manage semester courses. The system allows the user to add a course by entering the course code, course title, and credit unit. It can display all saved courses, search for a course by its code, calculate the total number of credit units, save courses to a file, and load previously saved courses from a file.

The application is implemented as a text-based console menu in Main.java. It uses Course and CourseManager classes to store course records in an ArrayList and perform all required operations through a numbered menu displayed in the terminal.

2. How Each Major Feature Works

- **Add Course:** The user selects option 1 from the menu and is prompted to enter a course code, title, and number of credit units. The input is validated (non-empty strings, positive integer units) and the course code is normalized to uppercase. Duplicate course codes are rejected. Valid courses are stored as Course objects inside the CourseManager ArrayList.
- **View All Courses:** When option 2 is selected, the program iterates through the ArrayList and displays each course in a formatted table with columns for code, title, and units. If no courses have been recorded, the system displays an appropriate message.
- **Search Course by Code:** The user selects option 3 and enters a course code. The system uses a recursive search method that checks each course in the list one at a time until a match is found or the list is exhausted. If a match is found, the course details are displayed; otherwise, a not-found message is shown.
- **Compute Total Units:** Option 4 triggers a recursive method that sums up the credit units starting from the last course and working backward to the first. The total number of courses and the total credit units are then displayed on the console.

- **Save to File:** Option 5 writes all courses to a text file in the format CODE|TITLE|UNITS, with one course per line. The user is prompted for a filename; if none is entered, the default file courses.txt is used.
- **Load from File:** Option 6 reads a text file line by line, validates each record, reconstructs Course objects, and replaces the current in-memory course list with the loaded data. If a line has an invalid format, the program throws an IOException with the line number.
- **Exit Program:** Option 7 ends the session. The program displays a goodbye message and closes the Scanner before terminating.

3. Challenges Encountered and Error Handling

Several challenges were addressed through exception handling and input validation:

- Empty or invalid menu choices are caught using try-catch blocks around Integer.parseInt(), with clear error messages prompting the user to retry.
- Duplicate course codes throw an IllegalArgumentException when adding a course, preventing data inconsistency.
- File operations use try-catch to handle FileNotFoundException (missing file), IOException (read/write failures), and malformed file records.
- String inputs are trimmed and course codes are converted to uppercase using toUpperCase() for consistent searching and comparison.
- Loading a file with missing or malformed fields throws an IOException with a descriptive line-number message.
- Empty input for course code, title, or units is rejected through validation loops that ask the user to re-enter the value until valid data is provided.

4. Where Recursion was Applied

Recursion is used in three places within the application:

- **Menu Loop (runMenu):** After each operation (except Exit), runMenu() calls itself to redisplay the menu and wait for the next choice. A depth counter prevents infinite recursion.
- **Course Search (recursiveSearch):** When searching by code, the method checks the course at the current index; if it does not match, it recursively checks the next index until the list ends.
- **Total Units Calculation (recursiveSumUnits):** The total credit units are computed by adding the units at the current index to the recursive sum of all preceding courses, with a base case of $\text{index} < 0$ returning 0.

Screenshot of the program running

```
Windows PowerShell
PS C:\Users\USER\Desktop\New folder\submission\source-files> cd "C:\Users\USER\Desktop\New folder\submission\source-files"
PS C:\Users\USER\Desktop\New folder\submission\source-files> javac *.java
PS C:\Users\USER\Desktop\New folder\submission\source-files> java Main

=====
STUDENT COURSE MANAGEMENT SYSTEM
=====

----- MENU -----
1. Add Course
2. View All Courses
3. Search Course by Code
4. Compute Total Units
5. Save to File
6. Load from File
7. Exit Program
-----

Enter your choice (1-7): 6

--- Load from File ---
Enter filename [courses.txt]:

Courses loaded successfully from 'courses.txt'.
Loaded 4 course(s).

----- MENU -----
1. Add Course
2. View All Courses
3. Search Course by Code
4. Compute Total Units
5. Save to File
6. Load from File
7. Exit Program
-----

Enter your choice (1-7): 2

--- All Courses ---

+-----+-----+-----+
| Code | Title | Units |
+-----+-----+-----+
| COS201 | Programming 1 | 3 |
| ENT211 | Entrepreneurship | 2 |
| SEN201 | Intro to Software Engineering | 3 |
| MTH205 | Linear Algebra 1 | 3 |
+-----+-----+-----+

Total courses: 4
```